

Bcrypt vs. Argon2: A Comparison and Their Impact on Digital Systems

Michael DiNapoli

School of Computing, Binghamton University

CS 301: Ethical, Social, and Global Issues in Computing

Professor Schiesser

May 10, 2026

Abstract

Passwords remain a central form of authentication for modern systems, but their effectiveness depends on hashing algorithms and user behavior. This paper compares bcrypt and Argon2, focusing on their resistance against modern password attacks. While bcrypt enhances security by salting passwords and increasing computational cost, its memory usage is limited, making it vulnerable to hardware-accelerated attacks. Argon2 increases computational cost and memory usage by using a memory-hard design that makes brute-force attacks more difficult. As a result, Argon2 provides stronger protection against modern attacks. In addition to technical uses, this paper examines the impact of hashing algorithms on the security of users and organizations. Poor password practices, such as reuse and low complexity, contribute to data breaches, identity theft, and financial loss. These issues can reduce user trust in web applications that fail to protect sensitive personal information. Overall, Argon2 offers a more secure hashing function than bcrypt, but effective security requires strong algorithms and good user behavior.

Keywords: Argon2, bcrypt, password, hashing, function, hardware

Bcrypt vs. Argon2: A Comparison and Their Impact on Digital Systems

Passwords remain one of the primary methods for people to log in to online services. Almost all web applications require a username and a password to verify a user's identity. Keeping passwords safe has become a major issue in cybersecurity because of the widespread use of passwords. For the most part, when companies store passwords, the passwords are not stored in plaintext (human-readable/unencrypted). The companies store a hashed version of the password by using a cryptographic hashing algorithm. Password hashing is a process where the hashing algorithm takes an input, a password in this case, and runs it through a function that produces an output called a hash. Each hash is unique to the input that created it, so even a small change will cause a difference (Biryukov et al., 2016, p. 292). Unique hashes help to protect user data in the case that a database containing usernames and passwords is stolen or accessed by a threat. Although this is a good defense, it has a weakness; hackers are able to retrieve passwords by using brute-force attacks or dictionary attacks. These attacks have a computer repeatedly guess passwords until the computer finds one that matches a stored hashed password. As technology has advanced, attackers take advantage of stronger Graphics Processing Units (GPUs) and specialized hardware, which allow attacks to be faster and more effective (Biryukov et al., 2016, p. 292). Every day, more and more people rely on the internet, and authentication becomes a greater problem. Progression from legacy hashing algorithms is necessary due to advancements in modern attacks using GPUs and ASICs (Tippe & Berner, 2025, pp. 25-26). As attackers advance, individuals and companies must also take necessary precautions to protect themselves against identity theft.

To prepare for these attacks, companies use password-hashing algorithms that are made to slow hackers down. Bcrypt is a widely used hashing algorithm designed to make password

cracking more difficult by adding additional computational cost and random salts. This algorithm was originally developed from the Blowfish encryption algorithm (a symmetric-key block cipher) to improve password security by increasing the time required to crack hashes. bcrypt combines a random salt with the password hash, which helps prevent attacks that rely on precomputed lookup tables, such as rainbow tables. A salt is random data added to existing data before it is hashed, so that even when two people have the same password, each has a unique hash (Batubara et al., 2021, pp. 1-2). However, while bcrypt is still widely used, advances in hardware have made it easier for attackers to perform a high volume of password guesses quickly. Hardware such as FPGAs, multiple-core GPUs, and ASICs accelerate password attacks when computational costs do not require extensive memory (Biryukov et al., 2016, p. 292). Therefore, bcrypt provides meaningful protection, but is less resistant than memory-hard algorithms such as Argon2 against large-scale hardware password attacks (OWASP Foundation, n.d.).

Newer password hashing algorithms have been designed to resist modern attacks that use specialized hardware. Argon2 is a modern hashing algorithm with a memory-hard design, meaning it requires significant amounts of memory and processing power when computing hashes. This method increases the cost of brute-force password attacks. Argon2 allows the system designers to adjust memory size, iteration count, and parallelism, which gives it more flexibility than bcrypt (Biryukov et al., 2016, p. 294). Compared to bcrypt, Argon2 provides a stronger defense against hardware-accelerated password attacks because it combines computational cost with memory cost.

Technical Description of Background Algorithm

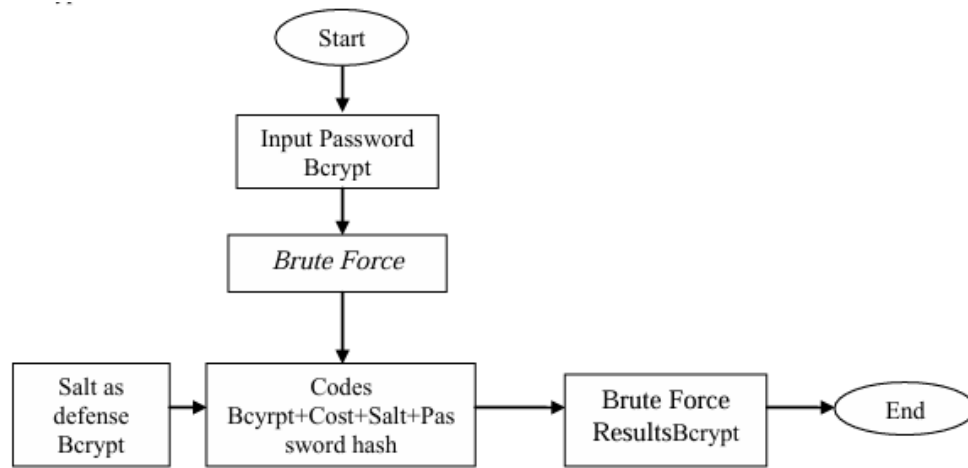
The bcrypt algorithm is designed to protect stored passwords from brute-force attacks. Bcrypt was developed by Niels Provos and David Mazières and is derived from the Blowfish encryption algorithm (Batubara et al., 2021, p. 1). Bcrypt was created to improve password security by making it more expensive for hackers to guess passwords. Unlike basic hashing algorithms such as MD5 and SHA-256, bcrypt is made to be slow and adaptable, which makes it protective against modern password-cracking methods.

Broadly speaking, bcrypt works by taking a user's passwords as input and generating a secure hash that can be stored in a database. When a user enters a password to log in, the password is hashed and then compared to the database to see if there is a match. If the hash is matched, the user is authenticated. This process relies on the fact that the same input will always output the same unique data when hashed, while still making the hashed password harder for a hacker to crack (Batubara et al., 2021, p. 2).

One of the most important features of bcrypt is the use of a salt. The purpose of the salt is to make sure that if two users or a hacker guess passwords from a table that are the same, the hash will still be unique to each user and differ from the hacker's guesses. The salt makes cracking passwords much more difficult for attackers to use precomputed lookup tables (rainbow tables) to steal passwords in plaintext from a database (Batubara et al., 2021, p. 2). The salt is stored with the hash, so it can be used during the authentication process. As shown in Figure 1 below, the process of brute-force attacks involves using an input password and hashing it (using bcrypt) with a salt and cost factor, which is followed by comparing the results of the brute-force attempts to stored hashes.

Figure 1

Bcrypt Brute Force Password Process Flow



Note. Adapted from Batubara (2021).

Another important feature of bcrypt lies in the cost factor, which controls how much computation is needed to create a hash. The cost factor determines how many times the hashing process is repeated, making the algorithm slower as the cost increases. High costs remain as a negative for hackers because they will need more capital to execute the attack, and computations will take longer. Cost factor is important because even as hardware improves, increasing the cost factor can defend against this. As GPUs and specialized equipment become faster, engineers can increase the cost factor to keep their databases secure (Batubara et al., 2021, p. 3). This ability to adapt to new hardware is one reason bcrypt has been used by many over time.

The internal process of bcrypt, based on the Blowfish cipher, involves multiple rounds of encryption and key expansion. During the hashing process, bcrypt takes three inputs to create the hash: the password, salt, and cost factor. These are used to get the algorithm started and to repeat the process many times to output the final hash into a single unique string (Batubara et al., 2021, p. 3). This structure allows systems to easily store and verify passwords input by users.

Although bcrypt strength is demonstrated against many attacks, it has limitations. One issue with bcrypt is that it only increases computational cost, while it does not cost a lot of memory. Attacks using hardware like GPUs can still perform many password guesses, which reduces the effectiveness of bcrypt over time (Biryukov et al., 2016, p. 292). This limitation becomes more significant as hardware improves.

Results from experiments done by Batubara et al. show both the strengths and weaknesses of bcrypt when brute-force attacks are used on it. For example, passwords consisting of only alphabetic characters and a length of four were cracked in four days, while longer passwords took longer than five days to crack. Similarly, numeric passwords with seven digits were cracked in eight to ten hours, which shows that a simpler password structure is more vulnerable (Batubara et al., 2021, p. 6). Passwords that mixed letters and numbers were harder to crack and were not found during the testing period, which demonstrates that the complexity and length of passwords play a role in their security. To understand how password complexity affects computational time, the results of testing different password types are summarized in Table 1 below.

Table 1

Brute Force Results of Bcrypt on Different Password Types

Character Type	Length	Result	Time
Alphabetic	4	Found	4 Days
Alphabetic	5	Not Found	5 Days
Numeric	7	Found	8-10 Hours
Mixed	7	Not Found	5 Days

Note. Adapted from Batubara (2021).

The results in Table 1 show that bcrypt is effective given the right parameters, such as a complex password, but due to human error (providing weak passwords), the hashing algorithm is not completely immune to attacks. On top of this, bcrypt does not increase memory usage in a way that is significant to the hacker.

Overall, bcrypt remains important and widely used for protecting passwords because it uses salts and adjustable cost factors. These features provide strong protection against brute-force and lookup tables. However, bcrypt mainly increases computational cost rather than memory cost, which limits its ability to protect against specialized hardware attacks. This limitation is one reason modern algorithms like Argon2id are preferred.

Technical Description of Primary Algorithm

Argon2, a more modern password-hashing algorithm, was designed to address the weaknesses of algorithms made before it, such as bcrypt. It was made by Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich (Biryukov et al., 2016, p. 292). The main purpose of Argon2 is to make password cracking much more difficult by requiring high computational power, like bcrypt, and large amounts of memory. This design makes the algorithm more resistant to attacks that use GPUs, ASICs, or other hardware used for attacking.

At a broad level, Argon2 works similarly in the way that it takes a password as input and produces a hash that is then stored in a database. However, the key difference is that Argon2 uses a memory-hard function, which means it requires a large amount of memory to compute each hash. This detail is very important because modern attacks rely on parallel hardware, such as GPUs, which are very efficient at performing many computations at once, but are limited by their available memory. Argon2 reduces the advantage this hardware has by forcing it to use much more memory compared to other algorithms (Biryukov et al., 2016, p. 292).

Argon2 uses several inputs when hashing a password. These include the password, a salt, and additional parameters that control how the algorithm runs. These additional parameters control parallelism, memory size, and the number of iterations (Biryukov et al., 2016, p. 294). Unlike bcrypt, whose main function is to slow password cracking by slowing down computation, Argon2 allows security engineers to adjust both computation time and memory usage separately. This makes it better suited for defending against modern attacks.

Internally, Argon2 involves filling a large block of memory using data derived from the password and salt. The algorithm processes the memory over and over using a compression function, which alters the data in a way that makes it difficult to predict or reverse. As shown in Figure 2, Argon2 organizes memory into blocks and updates them over multiple iterations using the compression function G .

Figure 2

Argon2 Memory Block Processing Model

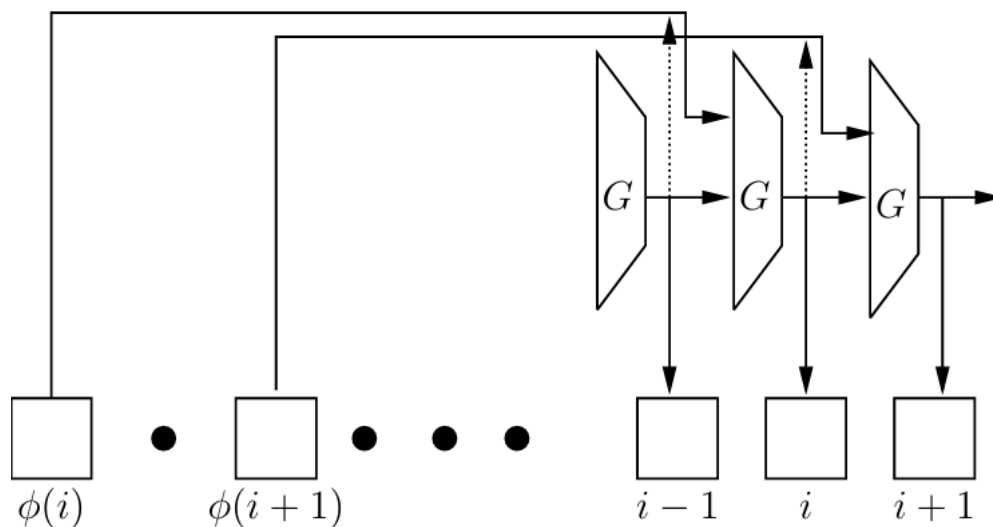


Figure 2 shows how memory blocks rely on previous memory blocks. Biryukov et al. (2016) show that G is a compression function that combines the memory blocks, and Φ is the indexing function that chooses which of the previous memory blocks are selected for the next

memory block. The resulting memory blocks, which came from the original inputs, are used to generate the final hash by passing it through a final hash function H (Biryukov et al., 2016, pp. 294-295).

Argon2's memory-dependent structure makes time-memory tradeoff attacks harder, but side-channel resistance depends on the variant. Argon2i and Argon2id are preferred when side channel attacks are an issue. If an attacker tries to reduce memory usage, they would be forced to compute additional computations, which would increase the cost of the attack. This feature is known as a tradeoff, and Argon2 is designed to make these trade-offs inefficient (Biryukov et al., 2016, p. 293).

Another important feature of Argon2 is that it has different variants, known as Argon2d, Argon2i, and Argon2id. Argon2d utilizes data-dependent memory access to provide greater resistance to time-memory tradeoff attacks, but is more vulnerable to side-channel attacks. Argon2i uses memory access that is data-independent, which improves side-channel resistance, but is slower (Biryukov et al., 2016, p. 294). These variants allow developers to choose the version that best fits their password security needs. A third variant, Argon2id, combines both approaches by using data-independent memory access in its early stage and data-dependent access afterward. Argon2id is the variant that is most commonly recommended for password hashing today (OWASP Foundation, n.d.).

Argon2's main strength comes from limiting the effectiveness of hardware-accelerated attacks. Modern GPUs perform many computations at the same time, which reduces the effectiveness of older algorithms like bcrypt. Argon2 addresses this by requiring a vast amount of memory per hash, which doesn't scale well for GPU attacks because of the cost (Biryukov et

al., 2016, p. 292). Requiring a large amount of memory makes large-scale attacks much more expensive and causes them to be less efficient.

Research done by Tippe and Berner (2025) shows that the adoption of Argon2 is increasing in modern software systems due to its resistance to attacks and its flexibility in adjusting security parameters (Tippe & Berner, 2025, pp. 26-27).

The equation below represents a basic model of Argon2 and how it processes memory blocks during the hashing process:

$$B[j] = G(B[\Phi_1(j)], B[\Phi_2(j)], \dots, B[\Phi_k(j)]). \quad (1)$$

In Equation 1, $B[j]$ is the current memory block, and $B[\Phi_1(j)], B[\Phi_2(j)], \dots, B[\Phi_k(j)]$ are the previous memory blocks selected from the memory using indexing functions Φ . The compression function G combines these blocks to create a new value, which becomes the current memory block (Biryukov et al., 2016, p. 293-294).

Overall, Argon2 improves on earlier password-hashing algorithms by using a memory-hard design as well as adjustable parameters. This provides stronger protection against modern attacks. This combination makes it much more difficult to efficiently use modern hardware for attacks, such as GPUs, to perform vast brute-force attacks.

Analysis

When comparing Argon2 and bcrypt, one must look at both hashing functions' ability to increase the difficulty of brute force password attacks to get through. Both algorithms were designed to protect against password attacks, but Argon2 provides stronger overall defense because of its memory-hard design and configurations for different attacks.

Table 2 shows a comparison between Argon2 and bcrypt. The table highlights Argon2's higher resistance to attacks due to its high memory usage.

Table 2*Comparison of Bcrypt and Argon2*

Feature	Bcrypt	Argon2
Memory Usage	Low	High
Computational Cost	Adjustable	Adjustable
Attack Resistance	High with proper configuration, but weaker against hardware-accelerated attacks	High when configured properly
Year	1999	2015

The most significant limitation of bcrypt is that it mostly increases the computational cost of an attack, while memory use is barely affected. Modern attacks often rely on hardware such as GPUs, FPGAs, and ASICs, which allow attackers to perform multiple computations at the same time (Biryukov et al., 2016, p. 292). Because of this, hackers can take advantage of bcrypt with parallel attacks. As hardware improves over time, the effectiveness of bcrypt decreases because the hardware can do more computations in a shorter amount of time.

Results from Batubara et al. (2021) highlight the limitations of bcrypt under brute-force attacks. Their findings show that simple passwords can be cracked in a relatively short time. This weakness demonstrates that the effectiveness of bcrypt depends on the complexity of the passwords and that certain password types are more vulnerable to attacks. The results in Table 1 make it clear that hashing alone does not guarantee strong security when passwords lack enough complexity (Batubara et al., 2021, p. 2). This shows a major weakness; the effectiveness of bcrypt depends on the input of a user, which is essentially random without password policies.

Argon2 addresses the weaknesses of bcrypt (that it minimally affects memory usage). It does this by using a function that relies on memory usage. Because of this, Argon2 “imposes

prohibitive time-memory and computation-memory tradeoffs on memory-saving users” (Biryukov et al., 2016, p. 292). This means attackers must choose between faster but costlier attacks or longer but cheaper attacks; either choice would make a large-scale brute force attack to be less effective.

Argon2 also provides strong protection against time-memory tradeoff attacks, which is another advantage. As explained previously, reducing memory usage increases the computation time significantly. This feature limits the effectiveness of specialized and modern hardware used in attacks (Biryukov et al., 2016, p. 294). This part of Argon2 solves the problem of modern hardware, which is used to accelerate brute force attacks.

Despite its advantages, Argon2 has its limitations. While the memory-hard design costs more for attackers, it also requires more memory and computations for the host compared to algorithms like bcrypt. Argon2 requires a specified amount of memory to be allocated (Biryukov et al., 2016, p. 292). This means that allocating too little memory can reduce the effectiveness of the hash, and allocating too much may reduce efficiency. However, when the configuration is optimal, these tradeoffs provide a stronger protection against attacks.

While making technical considerations, we must also consider how the effectiveness of both algorithms is affected by user passwords and advancing threats. Attackers use lists that contain vast amounts of the most common passwords and passwords that have been found in previous data breaches. This becomes a problem for users who reuse their passwords, which, according to Cranor (2014), often happens because users struggle to manage many different passwords across multiple applications. Das et al. (2014) provide strong support for this issue, estimating that 43-51% of users reuse the same passwords across multiple websites. This becomes an issue for password hashing algorithms because attackers will have a higher

probability of obtaining a user's credentials when they are used across multiple sites. While this is happening, the internet faces an increasing number of threats and more advanced threats. User behavior contributes to the vulnerability of personal information, often because of weak passwords (Zave & Rexford, 2020, p. 9). Together, these factors show why hashing algorithms are needed and that their effectiveness depends on the input.

While bcrypt is still used in many applications today, its effectiveness decreases over time against modern attacks. Argon2's approach to hashing provides a more secure method of protection against today's attacks, making it a stronger defense when configured properly.

Social Impact

Password Security has a substantial impact on almost all internet users. The impact grows as the internet expands and becomes more included in daily activities. Weak password policies and practices have been contributors to data breaches, which result in identity theft for users and financial loss for the company. Even though the advancements in hashing algorithms make them stronger, their effectiveness is heavily influenced by password complexity, which is up to the user.

The implications of weak password security are beyond the users. While users suffer from stolen personal information, companies that fail to protect data face legal consequences, financial losses, and damage to their reputation. When a database of passwords is stolen, the passwords will most likely be hashed. Attackers can perform brute-force or dictionary attacks by hashing candidate passwords and comparing them to stolen password hashes. This works especially well against simple passwords. As shown in Table 1, even secure hashing algorithms like bcrypt can be vulnerable to an attack when the complexity of a password is low, which increases the risk of stolen data.

To mitigate these risks, stronger hashing algorithms, such as Argon2, were developed. Argon2 decreases the effectiveness of brute-force attacks and specialized hardware by requiring more memory and computation. After a data breach, Argon2 is better suited to prevent attackers from recovering plaintext passwords. If systems are poorly configured or users continue using poor passwords, security will remain a human and organizational problem, not a technical one.

The effect of password security on public trust in digital systems has important implications for social impact. When user credentials are stolen, users often lose the ability to trust an organization to protect their personal data. Research shows that consumer trust can significantly decrease after a data breach results in data theft, even when the user chooses to use similar applications elsewhere (Strzelecki & Rizun, 2022, p. 1). Password reuse increases the likelihood of data theft because a compromised password that is used across multiple accounts threatens the users' accounts (Das et al., 2014, p. 1). As a result of this, users may reject applications when asked to share personal information. The decrease in trust has effects on an organization and society as a whole. Organizations must work to secure their systems and rebuild trust, while users must be conscious of their security.

To address the social impact of password security at a high level requires a change in both user and organizational choices. The user's job is to maintain their own security, but many continue to create and reuse the same weak passwords. At the same time, organizations must implement strong security policies and design systems that encourage good user behavior, such as password policies. Zave and Rexford (2020) emphasize that user choices, such as weak passwords or clicking unsafe links, can contribute to security vulnerabilities, but software bugs and poor system design play a major role. This shows that meaningful security depends on the user and system designers together.

Weak password security can lead to widespread consequences, including identity theft, financial loss, and reduced trust in companies. Over time, more aspects of life become digitized. Things like banking, communication, and healthcare are online, leaving your information vulnerable unless the right systems are put into place. Organizations have the ethical responsibility to protect their users' data by implementing security measures that minimize harm done by attackers. While the organizations maintain secure systems, the users must also be mindful of their security by using complex passwords. To have a significant impact on society, improvements to security will take time and cooperation from users and companies. Without these improvements, risks from password security will continue to grow along with the internet.

Conclusion

Passwords remain one of the most utilized forms of authentication despite their drawbacks. Password security continues to be challenged by attacks and hardware that keep advancing. While bcrypt is widely used and improves security by adding random salts and adaptable computation cost, its memory usage makes it susceptible to attacks utilizing modern hardware to accelerate password cracking. Argon2 uses a memory-hard design that significantly increases the cost and difficulty of brute-force attacks accelerated by modern hardware. As a result, Argon2 is a more secure password hashing algorithm.

On top of technical uses, password hashing functions have a significant effect on society's trust in web applications. When security systems fail due to poor implementation, weak passwords, policies, or human errors, private data is exposed, and the credibility of a company is damaged. Poor security systems lose the users' confidence in trusting online services for important activities such as banking and healthcare. For this reason, computing professionals must implement strong hashing algorithms and complete password standards to protect user data.

As digital systems grow, secure password hashing is essential not only for security but also for maintaining trust in online services that people use every day.

References

- Batubara, T. P., Efendi, S., & Nababan, E. B. (2021). Analysis performance BCRYPT algorithm to improve password security from brute force. *Journal of Physics: Conference Series*, 1811(1), 012129. <https://doi.org/10.1088/1742-6596/1811/1/012129>
- Biryukov, A., Dinu, D., & Khovratovich, D. (2016). Argon2: New generation of memory-hard functions for password hashing and other applications. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)* (pp. 292–302). IEEE. <https://doi.org/10.1109/EuroSP.2016.31>
- Cranor, L. F. (2014, March). *What's wrong with your pa\$\$w0rd?* [Video]. TED. <https://www.youtube.com/watch?v=0SkdP36wiAU>
- Das, A., Bonneau, J., Caesar, M., Borisov, N., & Wang, X. (2014, February). The tangled web of password reuse. In *Ndss* (Vol. 14, No. 2014, pp. 23-26). <https://www.covert.io/research-papers/security/The%20Tangled%20Web%20of%20Password%20Reuse.pdf>
- OWASP Foundation. (n.d.). *Password storage cheat sheet*. OWASP Cheat Sheet Series. Retrieved June 24, 2026, from https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- Strzelecki, A., & Rizun, M. (2022). Consumers' change in trust and security after a personal data breach in online shopping. *Sustainability*, 14(10), 5866. <https://www.mdpi.com/2071-1050/14/10/5866>

- Tippe, P., & Berner, M. P. (2025). Evaluating Argon2 adoption and effectiveness in real-world software. In M. Dalla Preda, S. Schrittwieser, V. Naessens, & B. De Sutter (Eds.), *Availability, Reliability and Security: ARES 2025* (Lecture Notes in Computer Science, Vol. 15993). Springer. https://doi.org/10.1007/978-3-032-00627-1_2
- Zave, P., & Rexford, J. (2020). Patterns and interactions in network security. *ACM Computing Surveys (CSUR)*, 53(6), 1-37. <https://dl.acm.org/doi/10.1145/3417988>